

Analysis: Catch Some

Test set 1 (Visible)

Firstly, observe that if Bundle switches the color of her shirt then she should never wear a previously worn color again as that would not give us a better answer. Also, for dogs of the same color, if she observes one dog then she should have observed all dogs who are at a position less than that dog. The condition that she doesn't need to return to her home after observing the last dog is a little tricky. Let's first solve the problem where she must return home after observing the last dog.

For this simplified version, we notice that the order of the color of shirts that Bundle has to put on doesn't matter. This gives rise to the following dynamic programming approach.

Let $dp[i][j]$ denote the minimum time Bundle needs to observe j dogs such that she has either observed or has decided she'll not observe dogs of colors 1 to i .

Hence, $dp[C][K]$ becomes our answer, where C is the total number of different colors.

To calculate $dp[i][j]$ we loop upon the number of dogs Bundle could have observed for the $(i-1)$ th color, and take the minimum cost among them. That is,

$$dp[i][j] = \min(dp[i-1][j] + 0, dp[i-1][j-1] + 2 * X_{i,1}, dp[i-1][j-2] + 2 * X_{i,2}, dp[i-1][j-3] + 2 * X_{i,3} \dots)$$

Where, $X_{c,d}$ represents the distance of the d -th dog of c -th color from Bundle's house.

Since the size of the loop for a particular color is the number of dogs of previous color + 1, the sum of sizes of all loops is $O(N)$. Hence, we have $O(N^2)$ states and do a total of $O(N)$ transitions. This gives us a run time of $O(N^2)$ for a given color as the last color.

To complete the solution, we simply do this considering every color as the last color and add in the cost of observing dogs of the last color separately. This will incur an additional $O(N)$ loop to iterate over all colors resulting in a final complexity of $O(N^3)$.

Test set 2 (Hidden)

We can optimize the previous solution to not do the same calculations with every color as the last color. This can be done by adding a boolean parameter in our state representing if we have already decided on the last color. We simply don't add the cost of returning back home whenever we set this parameter to true.

Essentially, we have $dp[i][j][k]$ where i and j still represent the same things and k is a boolean denoting if we have chosen the last color in the first i colors.

$$\text{Now, } dp[i][j][0] = \min(dp[i-1][j][0] + 0, dp[i-1][j-1][0] + 2 * X_{i,1}, dp[i-1][j-2][0] + 2 * X_{i,2} \dots)$$

$$dp[i][j][1] = \min(\min(dp[i-1][j][1] + 0, dp[i-1][j-1][1] + 2 * X_{i,1}, dp[i-1][j-2][1] + 2 * X_{i,2} \dots), \min(dp[i-1][j][0] + 0, dp[i-1][j-1][0] + X_{i,1}, dp[i-1][j-2][0] + X_{i,2} \dots))$$

Again we have $O(N^2)$ states and do a total of $O(N)$ transitions. But we just do this once and not once for every color, yielding a final complexity of $O(N^2)$.

Analysis: Circuit Board

Test set 1 (Visible)

Restating the problem, we are asked to find the largest sub-rectangle in a grid of integers such that the difference between the maximum and minimum integers in each row of the sub-rectangle is not more than K .

For test set 1, we have $K = 0$. This means we have to find the largest sub-rectangle in which all integers are equal in each of the rows.

To do this, we first define $P_{i,j}$ as the maximum number of contiguous cells on the right of the cell i, j which have the same value as cell i, j .

We immediately observe that $P_{i,j} = 1 + P_{i,j+1}$ if the integers in j and $(j+1)$ th columns of row i are equal, else $P_{i,j}$ is 1.

Once we have these values, our problem simply reduces to the well known problem of finding the area of the largest rectangle in a histogram. For every column c , we first divide it into segments where all integers are equal. For any such segment say from row r_1 to r_2 , we take $P_{i,c}$ for $i = r_1$ to r_2 as the heights of histogram. It is clear now that any rectangle in this histogram represents a rectangle where in all integers are equal in each row of the rectangle.

Calculating the largest rectangle in the histogram of width M can be done in $O(M)$ time using stacks. The idea is to find for every bar of the histogram, the closest bars to the left and right of the current bar which are shorter than the current bar. To find the closest shorter bar on the left (we can modify the same procedure to also find the closest shorter bar on the right), we traverse our histogram from left to right. We initially have an empty stack which we update as follows: If the current bar's height is greater than the top element of the stack, then it's the required bar for our current bar and we push the current bar's height into the stack. Otherwise we keep on popping the stack till we find a number which is lesser than the current bar's height (or the stack becomes empty, in which case there's no bar to the left which is shorter than the current bar). This top element now corresponds to our required bar. We then push the current bar's height into our stack. Since we push or pop at most once for every bar, the complexity of this procedure is linear as described above.

And hence, we'll have to spend $O(R)$ time per column with a total run time complexity of $O(RC)$.

Test set 2 (Hidden)

For this case, $K \geq 0$, so we re-define $P_{i,j}$ as the maximum length such that the difference between the maximum number and the minimum number present in all cells of row i with column $\geq j$ and $< j + P_{i,j}$ is not more than K .

To calculate $P_{i,j}$ we binary search for the first cell in the i th row and to the right of j th column such that the condition that the difference between the maximum and minimum number present in these cells is not more than K is violated. We can do this quickly if we can answer range minimum and range maximum queries quickly. Those can be dealt with a lot of different data-structures.

Once we have this, our problem again reduces to the problem of finding the area of the largest rectangle in a histogram. For, every column c , we take $P_{i, c}$ for $i = 1$ to R as the heights of histogram. It is clear now that any rectangle in this histogram represents a rectangle where in no row maximum and minimum elements differ by more than K .

If we use a data-structure like Sparse Table to answer the range minimum and maximum queries then we can find all the P-values in $O(RC \log(C))$ time. The last part of finding the largest area rectangle in histogram should be done once for each column, and hence takes $O(R)$ time for each case resulting in $O(RC)$ time in total. Combining both, we have a final complexity of $O(RC \log(C))$. We can further reduce this to a complexity of $O(RC)$ by replacing the Sparse Table data-structure with a two pointers variation using two dequeues, that approach is left as an exercise for the reader.

Analysis: Wiggle Walk

Test set 1 (Visible)

Approaching the problem naively, one might try to simply simulate what has been mentioned in the problem statement i.e. keep on moving the robot in the specified direction till it reaches an un-visited square. This approach is going to have a time complexity of $O(N^2)$, which although good enough for test set 1.

Test set 2 (Hidden)

The problem with the above approach is that it visits a lot of already visited squares which in worst case will be all the previously visited squares (Consider the input where you are given alternating E and W throughout). If we somehow get to the destination square for each instruction faster, we might be able to reduce the complexity.

Let's say the robot is in some row r and received an instruction W . Now, all the already visited squares (if any) it will pass before reaching an unvisited square have to form a contiguous interval in row r . This suggests that we may use intervals to represent all the visited squares in the same row.

With that in mind, consider we have a [set](#) of intervals for each row and each column of the grid to represent which cells have been visited in that particular row or column, let's call them interval-sets. Initially, all these sets are empty except for the set corresponding to row S_R , which has a single interval (S_C, S_C) , and the set corresponding to the column S_C , which has a single interval (S_R, S_R) .

Now, using this data-structure, let's try to find the destination square for the robot. Let's say it's in square (r, c) and got an instruction W . For this, first we search in the interval-set corresponding to row r . We will try to find which interval in this set contains c (there must definitely be one!). Once we find it, we immediately know what's going to be the new position for the robot! It's apparent that the same method works for all other directions as well.

All that remains now is to find a way to update our data-structure suitably to also include the newly visited square. This can be done in a very standard manner by simply finding the adjacent intervals for that square in both, the corresponding column interval-set and the corresponding row interval-set and then updating them either by extending one of the intervals or merging them or adding a new 1 length interval.

Since we add at most one interval in each case, the number of intervals is $O(N)$. Since all operations are about finding/inserting/removing a single interval, all of those can be handled easily in $O(\log(N))$ time. So the over all run time of this approach is $O(N \log(N))$. There is also a $O(N)$ solution to this problem using hash tables. It is left as an exercise to the reader.